

Lab Exercise 4- Secure Coding & Defensive Design

1. Objective

To implement **secure coding practices** in a REST API by:

1. Validating user inputs to prevent **injection attacks and resource exhaustion**
 2. Implementing **secure error handling** to prevent stack trace leakage
 3. Implementing **secure logging and monitoring** for incident response
 4. Testing the APIs using **Postman**
-

2. Software / Tools Required

Tool	Purpose
VS Code	Development environment
Java JDK 17+	Programming language
Maven	Build tool
Spring Boot	Backend framework
Postman	API testing

Recommended VS Code Extensions:

- Extension Pack for Java
- Spring Boot Extension Pack
- Maven for Java

3. Secure Coding Concepts Covered

Security Principle	Description
Input Validation	Prevent malicious or malformed inputs
Resource Control	Prevent excessive data submission
Secure Error Handling	Prevent stack traces from reaching users
Secure Logging	Log useful information without exposing secrets
Monitoring	Maintain logs for security analysis

4. Create Spring Boot Project

Open VS Code → Press

Ctrl + Shift + P

Select

Spring Initializr: Generate Maven Project

Project Metadata

Group: **com.example**

Artifact: **securecoding**

Name: **securecoding**

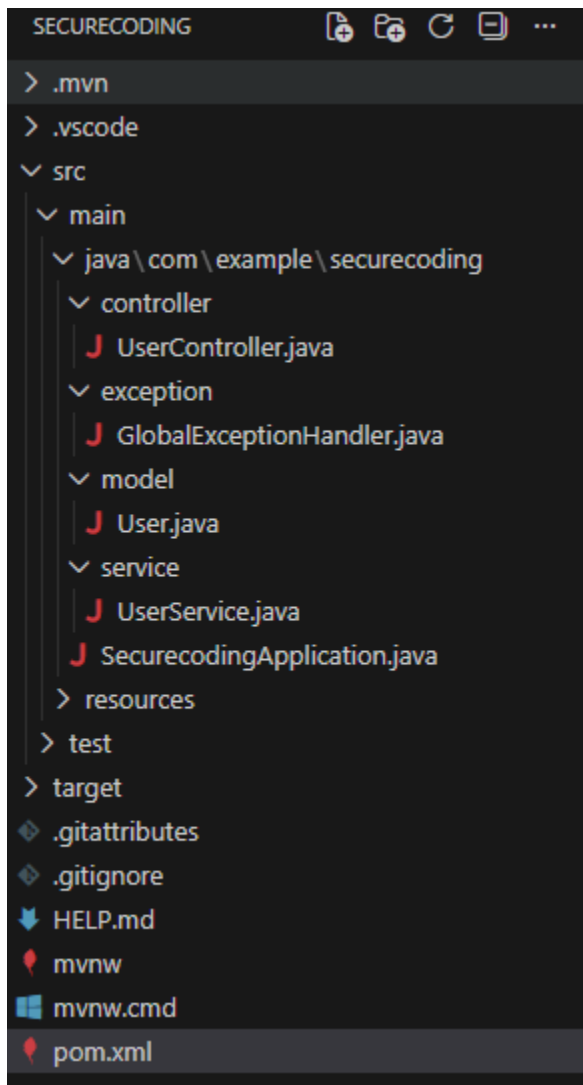
Packaging: **Jar**

Java: **17**

Dependencies

Spring Web

5. Project Structure



pom.xml

Dependencies

```
<dependencies>
  <!-- Spring Boot Web -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Input Validation -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>

  <!-- Logging (SLF4J + Logback included automatically) -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-logging</artifactId>
  </dependency>

  <!-- Development Tools -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>

  <!-- Lombok (Optional) -->
  <dependency>
```

```
<groupId>org.projectlombok</groupId>
<artifactId>lombok</artifactId>
<optional>true</optional>
</dependency>

<!-- Testing -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>

</dependencies>
```

6. Model Class – User

Path

model/User.java

```
package com.example.securecoding.model;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class User {

  private Long id;

  @NotBlank(message = "Name is required")
  @Size(max = 50, message = "Name too long")
  private String name;
```

```
@Email(message = "Invalid email format")
private String email;

@Size(min = 8, max = 20)
private String password;

public User() {}

public User(Long id,String name,String email,String password){
    this.id=id;
    this.name=name;
    this.email=email;
    this.password=password;
}

public Long getId(){ return id; }
public void setId(Long id){ this.id=id; }

public String getName(){ return name; }
public void setName(String name){ this.name=name; }

public String getEmail(){ return email; }
public void setEmail(String email){ this.email=email; }

public String getPassword(){ return password; }
public void setPassword(String password){ this.password=password; }
}
```

7. Service Layer

Path

service/UserService.java

```
package com.example.securecoding.service;

import com.example.securecoding.model.User;
import org.springframework.stereotype.Service;

import java.util.*;

@Service
public class UserService {

    private final Map<Long,User> database = new HashMap<>();

    public Collection<User> getAllUsers(){

        return database.values();

    }

    public User addUser(User user){

        if(database.size()>10){

            throw new RuntimeException("Resource limit exceeded");

        }

        user.setId((long)(database.size()+1));

        database.put(user.getId(),user);

        return user;

    }

}
```

8. Controller Layer (Input Validation)

Path

controller/UserController.java

```
package com.example.securecoding.controller;
import com.example.securecoding.model.User;
import com.example.securecoding.service.UserService;

import jakarta.validation.Valid;

import org.springframework.web.bind.annotation.*;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.util.Collection;

@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserService service;

    private static final Logger logger =
        LoggerFactory.getLogger(UserController.class);

    public UserController(UserService service){

        this.service = service;
    }

    @GetMapping
    public Collection<User> getUsers(){

        logger.info("Fetching all users");

        return service.getAllUsers();
    }
}
```

```
@PostMapping
public User createUser(@Valid @RequestBody User user){

    logger.info("Creating user with email: {}",user.getEmail());

    return service.addUser(user);

}

}
```

9. Secure Error Handling

Path

exception/GlobalExceptionHandler.java

```
package com.example.securecoding.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<String> validationError(){

        return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body("Invalid request data");

    }

    @ExceptionHandler(Exception.class)
```

```
public ResponseEntity<String> handleError(){  
  
    return ResponseEntity  
        .status(HttpStatus.INTERNAL_SERVER_ERROR)  
        .body("Internal Server Error");  
  
}
```

This prevents **stack traces from leaking to the client**.

10. Logging Configuration

File

application.properties

11. What to Log vs What NOT to Log

Log	Do Not Log
Login attempts	Passwords
User ID	Credit card numbers
API errors	Authentication tokens
Security events	Personal data

12. Running the Application

13. Testing APIs Using Postman

13.1 Create User

POST

`http://localhost:8080/api/users`

Body

```
{  
  "name": "Alice",  
  "email": "alice@test.com",  
  "password": "secure123"  
}
```

Expected Response

User created

13.2 Input Validation Test

Send invalid email

```
{  
  "name": "Bob",  
  "email": "invalid-email",  
  "password": "123"  
}
```

Response

Invalid request data

13.3 Resource Exhaustion Test

Try submitting many users quickly.

Expected

Resource limit exceeded

14. Secure Error Handling Test

Test Case — Send Null Body

POST

http://localhost:8080/api/users

Body

Leave body **empty**.

Expected

Internal Server Error

Test Case — Send Incorrect Data Type

Send a wrong type for ID.

```
{  
  "id":"text",  
  "name":"Alice",  
  "email":"alice@test.com",  
  "password":"secure123"  
}
```

Spring will throw a **conversion exception**, triggering:

```
Internal Server Error
```

15. Result

Thus, the **Secure Coding & Defensive Design principles** were successfully implemented using **Spring Boot APIs**, including:

- Input validation to prevent injection and abuse
- Secure error handling preventing stack trace leaks
- Secure logging practices for monitoring and incident response
- API testing using Postman